Software Agile Project Management Principle

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

Outline

- The fundamental and application of agile project principle.
- The concepts of project identification, analysis, treatment and monitoring.
- Documenting planning.
- User stories.

Fundamental and Application of Agile Project Principle

Learning Outcomes

By the end of this chapter, you will be able to explain the foundations of Agile, interpret the Agile Manifesto, and apply its principles in practice. You'll also learn how Agile principles encourage adaptability, collaboration, and delivering value in projects.

- Understand the fundamentals of Agile principles
- Explain the 12 Agile principles behind the manifesto
- Compare Agile with traditional project management
- Apply Agile concepts in software projects
- Recognize the benefits of collaboration and adaptability

What is Agile Project Management?

Agile Project Management is a modern approach that focuses on flexibility and iterative progress. Unlike predictive methods, Agile adapts to change and emphasizes customer value, collaboration, and continuous delivery.

- A flexible, iterative approach to project management
- Delivers value early and continuously
- Emphasizes collaboration, feedback, and adaptation
- Contrast: adaptive vs. predictive methods

Why Agile?

Agile emerged to address the shortcomings of traditional methods. Instead of waiting until the end, Agile ensures frequent delivery and constant collaboration with stakeholders, making projects more adaptive and customer-focused.

- Traditional approaches face:
  - Fixed scope, rigid planning, late delivery
- Agile responds with:
  - Flexibility to change at any stage
  - Small increments delivered frequently
  - Continuous stakeholder engagement

Agile Manifesto Overview

The Agile Manifesto is the foundation of Agile practices. It consists of four values and twelve guiding principles that shape how Agile projects are managed. These principles apply across different frameworks such as Scrum, Kanban, and Extreme Programming.

- Introduced in 2001 by 17 software experts
- Consists of:
  - 4 Core Values
  - 12 Guiding Principles
- Foundation of Agile methodologies (Scrum, Kanban, XP)

Agile Manifesto Overview

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Agile Value 1

The first value emphasizes that people and collaboration are more important than rigid tools and processes. Success comes from motivated individuals working together, not just following a checklist.

- Individuals and interactions over processes and tools
- People drive success, not tools alone
- Communication & teamwork create better outcomes

Agile Value 2

Agile prioritizes delivering working software rather than producing extensive documents. While documentation has value, the true measure of progress is functional, usable output.

- Working software over comprehensive documentation
- Documentation is helpful but secondary
- Functional results prove real progress

Agile Value 3

This value stresses the importance of working closely with customers throughout the project. Instead of relying only on contracts, Agile welcomes customer feedback to ensure the solution truly meets their needs.

- Customer collaboration over contract negotiation
- Involve customers in development
- Requirements evolve with feedback

Agile Value 4

Agile acknowledges that change is inevitable. Rather than rigidly following a fixed plan, Agile adapts and adjusts to meet evolving requirements and deliver real value.

- Responding to change over following a plan
- Plans adapt to new insights
- Flexibility creates better outcomes

Agile Principles Overview

The twelve Agile principles translate the four core values into practical actions. They provide a detailed roadmap for Agile teams to adapt, collaborate, and deliver effectively.

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Principle 1: Customer Satisfaction

The first principle emphasizes customer satisfaction by delivering value early and often. In Scrum, this is achieved through short iterations, as seen in the FBI Sentinel project where frequent demos kept stakeholders engaged.

- Priority: satisfy the customer
- Deliver early and continuously
- Scrum example: FBI Sentinel frequent demos

Principle 2: Welcome Change

Agile sees change as an opportunity, not a problem. Even late in development, adapting to new needs helps projects stay competitive. Scrum embraces this with sprint planning and backlog adjustments.

- Accept changes, even late in development
- Change = competitive advantage
- Scrum: sprint flexibility

Principle 3: Frequent Delivery

Instead of one big delivery at the end, Agile promotes frequent, small releases. This incremental approach ensures value is delivered regularly and feedback is gathered early.

- Deliver working software every 2–4 weeks
- Incremental releases improve value
- Scrum increments demonstrate progress

Principle 4: Collaboration

Agile promotes strong collaboration between developers and business stakeholders. Daily discussions improve alignment, decision-making, and team trust.

- Daily collaboration between business & developers
- Builds trust and faster decisions
- Scrum: daily standups

Principle 5: Motivated Individuals

Agile values motivated individuals, giving them the tools and trust to succeed. Scrum demonstrates this by empowering teams to self-organize and take ownership of their work.

- Projects succeed with motivated people
- Provide support & trust
- Scrum: self-organizing teams

Principle 6: Face-to-Face Communication

Communication is most effective face-to-face. Agile encourages team discussions to quickly resolve issues and ensure shared understanding, such as daily standups and retrospectives in Scrum.

- Best way to share information
- Builds stronger understanding
- Scrum: standups, retrospectives

Principle 7: Working Software as Progress Measure

Agile teams measure success by the software delivered, not by documentation produced. Delivering working software proves tangible progress.

- Primary measure = working software
- Not documents or reports
- Progress = usable product

Principle 8: Sustainable Pace

Agile emphasizes a balanced pace of work. By avoiding unrealistic deadlines, teams can maintain consistent productivity and quality.

- Agile promotes steady, sustainable pace
- Avoid burnout and fatigue
- Scrum: maintain constant velocity

Principle 9: Technical Excellence

Agile promotes technical excellence and strong design practices. By ensuring quality at every step, teams can adapt more easily and avoid costly rework.

- Quality & good design enhance agility
- Continuous attention to improvement
- Example: Definition of Done ensures quality

Principle 10: Simplicity

Agile emphasizes simplicity by focusing on essential features. The goal is to maximize the amount of work not done, while still delivering value.

- Do only what is needed
- Avoid unnecessary work
- Focus on value

Principle 11: Self-Organizing Teams

Agile recognizes that self-organizing teams produce the most effective outcomes. By empowering teams to decide how to work, they innovate and take full responsibility for delivery.

- Best results come from self-organizing teams
- Teams decide how to achieve goals
- Scrum: empowered development teams

Principle 12: Reflection & Adaptation

Continuous improvement is at the heart of Agile. Teams pause regularly to reflect and adapt their processes, ensuring better performance over time.

- Teams reflect regularly
- Adjust process to improve
- Scrum: retrospectives

Agile Mindset

The Agile mindset goes beyond frameworks. It's about adopting flexibility, continuous value delivery, and collaboration as part of the team culture.

- Focus on adaptability and collaboration
- Deliver continuous value
- Encourage experimentation and learning

Scrum as an Example of Agile in Action

Scrum is the most widely used Agile framework. It applies Agile values and principles through time-boxed sprints, defined roles, and regular ceremonies to ensure adaptability and delivery.

- Scrum applies Agile values & principles
- Short sprints (2–4 weeks)
- Roles: Product Owner, Scrum Master, Dev Team
- Events: Sprint, Standup, Retrospective

Summary of Agile Fundamentals

In summary, Agile is both a mindset and a set of guiding principles. Its values and principles promote adaptability, collaboration, and delivering customer value. Scrum is a strong example of how these principles are applied in practice.

- Agile = mindset + values + principles
- 4 Values + 12 Principles = Agile foundation
- Encourages adaptability, collaboration, and customer focus
- Scrum = practical application

Concepts of Project Identification, Analysis, Treatment

Why Identification & Monitoring are Critical

Project identification ensures that goals, scope, and risks are clear from the start. Monitoring then provides a feedback loop, helping teams stay aligned and adapt when challenges arise. Both are critical for Agile success.

- Identifies scope, risks, and resources early
- Ensures alignment with objectives and stakeholders
- Monitoring tracks progress & adapts plan as needed

Vision & Stakeholders

Every Agile project starts with a clear vision. Identifying stakeholders early ensures that their needs are considered, making it easier to align priorities and reduce misunderstandings.

- Define project vision – overall goal and purpose
- Identify key stakeholders – customers, users, sponsors
- Align expectations early to avoid conflicts

Techniques for Identification

Agile teams use tools like SWOT and stakeholder mapping to identify internal strengths and external risks, while also categorizing stakeholders by their influence and importance to the project.

- SWOT Analysis – strengths, weaknesses, opportunities, threats
- Stakeholder Map – influence and interest levels
- Helps in prioritizing stakeholder involvement

Project Analysis: Feasibility & Risk Assessment

Project analysis evaluates whether the project is worth pursuing. Agile promotes early feasibility checks and risk assessment to avoid wasted time and resources.

- Check technical, financial, and operational feasibility
- Assess risks using Agile-friendly methods
- Prioritize manageable and valuable requirements

Backlog Creation: Identifying Requirements

In Agile, requirements are stored in a dynamic backlog, not fixed documents. User stories make requirements easier to prioritize and adapt as new information emerges.

- Use Product Backlog to list features
- Requirements captured as user stories
- Continuously refined throughout project

MoSCoW Prioritization

MoSCoW is a prioritization method widely used in Agile. It helps teams focus on what is essential first, while leaving room for flexibility as time and resources permit.

- Must have – critical features
- Should have – important but not essential
- Could have – nice to include if possible
- Won't have (now) – excluded from this release

Timeboxing: Managing Effort & Risks

Timeboxing ensures that work is completed within strict time limits. This reduces the risk of scope creep and forces teams to focus on delivering the highest-value tasks within the available time.

- Fixed time periods for tasks (sprints, iterations)
- Encourages focus on priorities
- Reduces risk of endless delays

Monitoring with Burndown Chart

Burndown charts give teams a simple, visual way to track whether work is on schedule. If the trend shows slippage, corrective action can be taken quickly.

- Tracks remaining work vs. time
- Visual indicator of sprint progress
- Helps detect delays early

Kanban Board: Task Visualization

Kanban boards give teams a visual method of tracking task progress. By moving tasks across columns, bottlenecks are quickly identified, improving workflow efficiency.

- Columns: To Do → In Progress → Done
- Visualize work and bottlenecks
- Supports continuous flow of tasks

Retrospectives: Continuous Learning

Agile encourages regular retrospectives where teams reflect on their performance.

This continuous feedback loop drives learning and improvement over time.

- Held at the end of each sprint
- Identify what worked and what didn't
- Adjust processes for future improvement

Daily Standups: Team Synchronization

Daily standups keep the team aligned and quickly highlight blockers. This ensures everyone is on the same page and can collaborate to solve problems.

- 15-minute daily meeting
- Team answers 3 key questions:
  - What did I do yesterday?
  - What will I do today?
  - What's blocking me?

Case Example: FBI Sentinel Project (1/4)

The FBI Sentinel project initially struggled with Waterfall. Switching to Agile allowed faster delivery, cost savings, and improved stakeholder satisfaction.

- Large FBI project failed under Waterfall
- Restarted using Agile & Scrum
- Delivered faster at lower cost

Case Example: FBI Sentinel Project (2/4)

Scrum sprints and demos enabled the FBI team to deliver working increments regularly. This kept stakeholders actively involved and allowed the team to adjust quickly.

- Used short sprints for incremental delivery
- Frequent demos kept stakeholders engaged
- Improved adaptability to changes

Case Example: FBI Sentinel Project (3/4)

Burndown charts provided visibility of team progress. This transparency helped ensure accountability and reduced misunderstandings between stakeholders.

- Monitoring progress with burndown charts
- Clear visibility of completed vs pending work
- Reduced miscommunication

Case Example: FBI Sentinel Project (4/4)

Through retrospectives and empowered teams, the FBI Sentinel project became a success story, proving that Agile works even in large government organizations.

- Retrospectives improved process
- Motivated, self-organizing teams
- Success story for Agile in government

Case Example: Healthcare.gov Failure

Healthcare.gov's launch failure highlights the risks of rigid planning. Without Agile-style monitoring and iteration, the project could not adapt to problems in time, leading to a high-profile failure.

- Used traditional Waterfall approach
- Resulted in delays and technical failures
- Lack of iterative monitoring caused collapse

Section Wrap-up

This section showed how Agile helps identify, analyze, treat, and monitor projects effectively. With practical tools and real-world cases, Agile proves its ability to improve outcomes compared to traditional methods.

- Identification: vision, stakeholders, requirements
- Analysis: feasibility, prioritization
- Treatment: timeboxing, risk control
- Monitoring: burndown, Kanban, retrospectives
- Case studies: FBI Sentinel (success), Healthcare.gov (failure)

Documenting Planning

Agile Documentation vs Predictive Documentation

In traditional project management, teams spend months writing detailed documentation before work starts. Agile flips this approach by keeping documentation lightweight and adaptable, focusing on delivering real value through working software.

- Traditional (predictive) → heavy documentation upfront
- Agile → lightweight, living documents
- Focus on value over paperwork

Planning Levels in Agile

Agile planning occurs at multiple levels. It begins with the product vision, is broken into roadmaps and release plans, and finally refined in sprint plans, ensuring planning adapts across different time horizons.

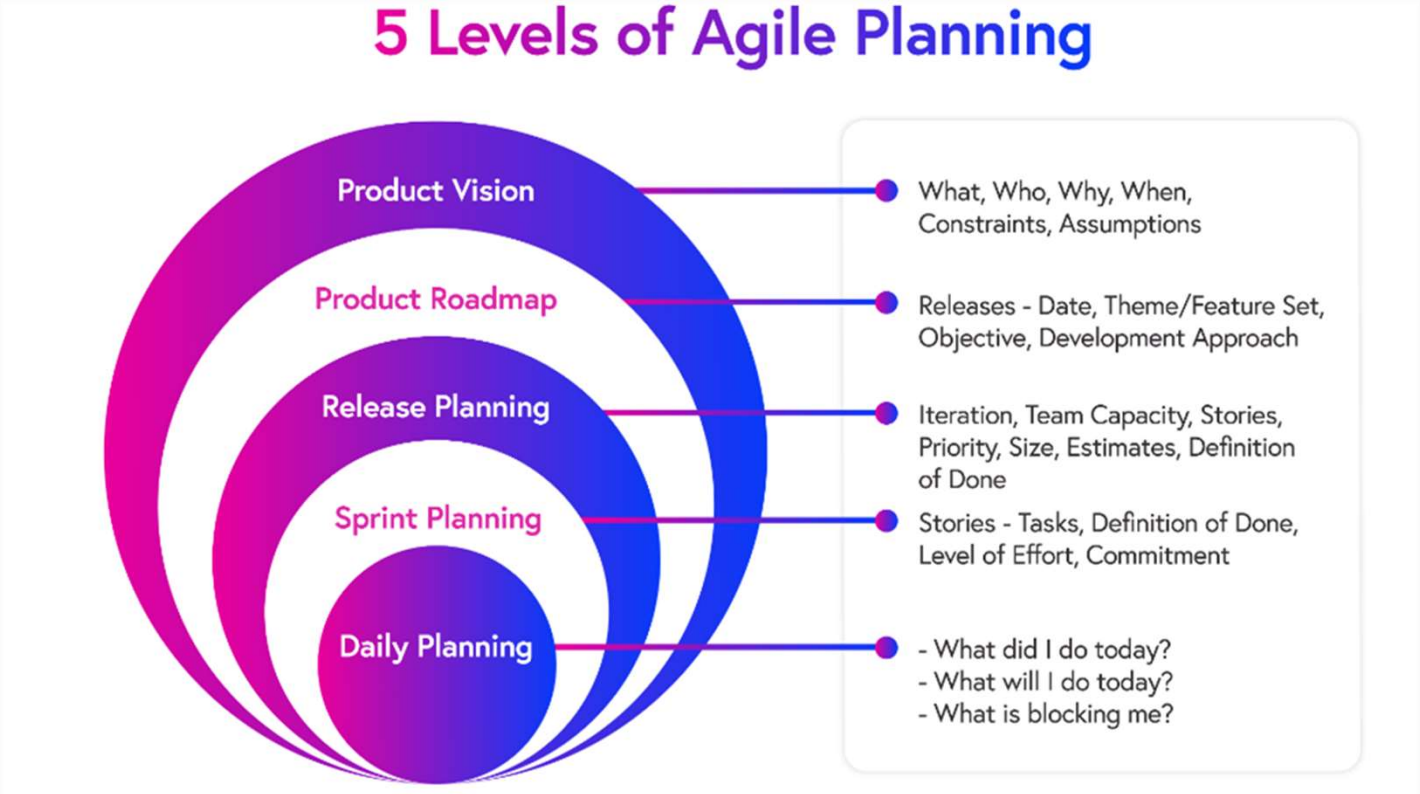
- Vision – long-term purpose of the product
- Roadmap – high-level timeline & direction
- Release plan – upcoming deliverables
- Sprint plan – short-term, detailed tasks

Planning Levels in Agile

Agile plans at multiple horizons. Vision, roadmap, release and sprint set direction; the daily plan is the operational heartbeat where the team re-plans work every day to protect the sprint goal—updating tasks, owners, and the board as reality changes.

- Vision – long-term purpose & outcomes
- Roadmap – high-level time-phased themes/epics
- Release plan – what fits into the next(s) release(s)
- Sprint plan – 1–4 week iteration goal & selected stories
- Daily plan – day-to-day re-planning to hit the sprint goal (Daily Scrum)

Planning Levels in Agile



Product Vision Document (Example)

The product vision defines the “why” of the project. A clear vision keeps the team aligned with long-term goals while avoiding scope drift.

- Describes overall product goal
- Aligns team & stakeholders
- Example: “Provide students with a seamless e-learning experience”

Roadmap (Example Visualization)

Agile roadmaps provide a big-picture view of the project's direction. Unlike rigid Gantt charts, they evolve with stakeholder input and changing requirements.

- Timeline view of major deliverables
- Shows releases, milestones, dependencies
- Keeps stakeholders informed

Release Planning in Agile

Release planning organizes the backlog into sets of features that can be delivered together. This ensures customers see continuous progress and value at regular intervals.

- Groups features into deliverable releases
- Ensures value delivered incrementally
- Balances scope, time, and quality

Sprint Planning (Scrum)

Sprint planning sets the direction for each iteration. The team chooses high-priority items from the backlog, defines a sprint goal, and breaks them into tasks that can be completed within the sprint.

- Held at start of each sprint
- Team selects items from product backlog
- Defines sprint goal + tasks

Daily Planning (Daily Scrum / Micro-Plan)

Daily planning happens in the Daily Scrum. The team inspects progress and immediately adapts the day's plan—adding/splitting tasks, re-assigning work, and surfacing blockers. It's not a status report; it's a micro-planning session to keep the sprint on track.

- Purpose: re-plan the day to meet the sprint goal
- Cadence: 15 minutes, same time/place, whole team
- Focus: yesterday, today, impediments (then adjust tasks)
- Updates: task board, WIP limits, burndown trend
- Outcomes: clear next steps, owners, escalated blockers

Product Backlog: Dynamic Documentation

The product backlog is the single source of truth for project requirements. It evolves constantly, making it a living document that adapts as priorities change.

- Master list of all requirements
- Written as user stories
- Continuously refined (backlog grooming)

Sprint Backlog: Living Plan

The sprint backlog defines the team's immediate work. Unlike static project plans, it changes as new insights appear during the sprint, ensuring flexibility while keeping accountability.

- Subset of product backlog
- Contains tasks for the current sprint
- Visible to the whole team

Definition of Done (Scrum Example)

In Scrum, a Definition of Done ensures that work is not just “finished” but actually meets quality standards. It avoids ambiguity and ensures deliverables are usable.

- Checklist of criteria for “done”
- Ensures quality and consistency
- Example: tested, documented, integrated

Kanban as Documentation Tool

Kanban boards act as living documentation of work. By visually tracking tasks, they replace long reports and provide transparency for all stakeholders.

- Visualizes work flow: To Do → In Progress → Done
- Documents real-time status
- Helps spot bottlenecks

Lightweight Planning: Avoiding Waste

Agile minimizes waste by producing only the documents that are actually useful. This keeps the team focused on communication and working software rather than endless paperwork.

- Plan only what is needed
- Avoid creating unused documents
- Emphasize communication over paperwork

Case Example: Spotify Model (1/3)

Spotify adopted Agile to manage rapid growth. Their squads (small, autonomous teams) relied on lightweight planning tools such as visual roadmaps and evolving backlogs instead of heavy documentation.

- Spotify built Agile around autonomous squads & tribes
- Minimal documentation, heavy use of visual tools
- Teams plan with roadmaps & backlogs instead of Gantt charts

Case Example: Spotify Model (2/3)

Spotify's planning model works at several levels. Squads plan sprints, tribes align them into broader goals, and the company sets a high-level vision. Roadmaps act as evolving guides, encouraging adaptability.

- Planning at multiple levels: squad → tribe → company
- Roadmaps used as guidelines, not fixed contracts
- Focus on adaptability and innovation

Case Example: Spotify Model (3/3)

Spotify emphasizes collaborative tools such as Kanban and backlog refinement. Documentation is limited to what helps teams align and adapt, ensuring efficiency without excessive paperwork.

- Strong use of Kanban & backlog refinement
- Documentation limited to what supports collaboration
- Teams empowered to adjust plans dynamically

Section Wrap-up: Planning as a Continuous Activity

In Agile, planning never stops. Teams set long-term intent, detail the near term, and re-plan daily to respond to change. Visible artifacts (backlogs/boards/DoD) and tight cadences keep everyone aligned while minimizing waste.

- Lightweight & evolving: vision → roadmap → release → sprint → daily plan
- Artifacts: backlog, boards, DoD, plans visible to all
- Practices: sprint planning + daily re-planning + reviews/retros
- Key idea: plan just enough now, refine continuously

User Stories

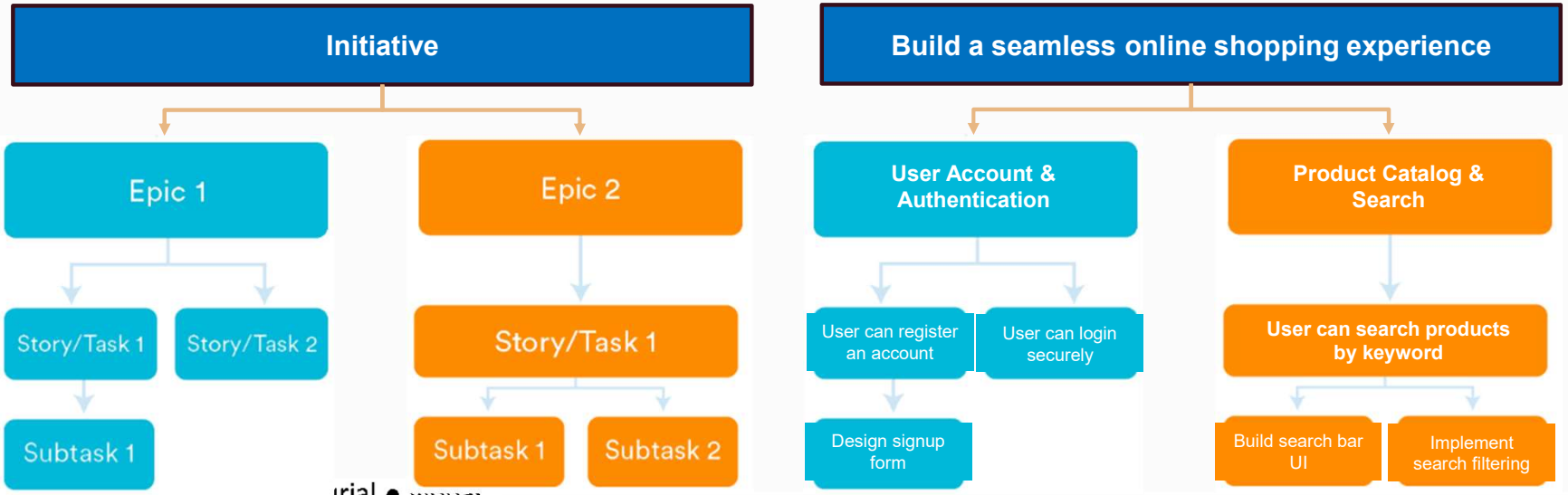
Introduction: User Story and Agile

- A user story is an informal, general explanation of a software feature written from the perspective of the end user. Its purpose is to articulate how a software feature will provide value to the customer.
- User stories are one of the core components of an agile program. it drives collaboration, clarity, and better products.
- Written in non-technical language, it keeps the end user at the center. It explains what is built, why, and the value for the customer.

Working with User Story

- **Scrum**: User stories go into sprints, driving estimation, planning, and forecasting.
- **Kanban**: User stories move through workflow, helping manage WIP and refine processes. In both, stories improve agility and efficiency.

User stories in a building block
Initiative → Epic → Story



Variations of User Story Format

The common format ensures every user story explains the who, what, and why.

Format	Structure	Example	Best Use Case
Role–Feature–Reason	*As a [role], I want [feature] so that [reason/benefit].*	*As a shopper, I want to save items in a wishlist so that I can purchase them later.*	Simple, user-focused; best for **capturing user needs quickly**.
Collaboration-based Approach	*As a [intended user]* *I want [intended action],* *so that [purpose/result of the action, profit achieved],* *Acceptance Criteria:*	*As a **customer of the bank*** *I want to **log into the system*** *So that **I can use bank products*** ***Acceptance criteria:*** • *Login must be a valid email address.* • *System rejects login attempt with incorrect password.* • *System rejects login attempt for non-existent user login.*	Ensures **clear testable conditions**; best for defining **acceptance criteria**.
Testable Format (BDD)	*Scenario → Given [initial context], When [event], Then [outcome].*	*Scenario: Add product to cart → Given I am on a product page, When I click "Add to Cart", Then the item appears in my cart.*	Best for **detailed testing** and collaboration between **devs, testers, and business**.

INVEST Criteria

The INVEST model provides six checks for quality user stories. A story that meets these standards is clear, actionable, and delivers value.

- Independent – can stand alone.
- Negotiable – open to discussion
- Valuable – delivers benefit
- Estimable – can be sized
- Small – manageable scope
- Testable – has acceptance criteria

Criterion	Example
Independent	*As a shopper, I want to reset my password so that I can regain access to my account.*
Negotiable	*As a user, I want to filter products by category so that I can find what I need faster.*
Valuable	*As a customer, I want to receive order confirmation emails so that I know my purchase was successful.*
Estimable	*As a user, I want to update my profile picture so that my account feels personalized.*
Small	*As a user, I want to add a product to my cart with one click.*
Testable	*As a customer, I want to see the total price including tax in my cart so that I know the full cost before checkout.*

Example of User Story
COLLABORATION-BASED APPROACH

User story US-002-02
As a logged-in bank customer
I want to be able to transfer money to another account
So that be able to transfer funds between accounts

Acceptance criteria
(AC1) the amount of the transfer cannot exceed the account balance
(AC2) target account must be correct
(AC3) for valid data (amount, account numbers) the source account balance decreases and the destination account balance increases by the amount of the transfer
(AC4) the transfer amount must be positive and represent the correct amount of money (i.e., be accurate to two decimal places max)

🔷 How it might look in Jira

Epic: Banking Transactions
Story: US-002-02 Transfer Money Between Accounts
Task: Frontend – Transfer Form
- Sub-task: Design form
- Sub-task: Implement form
- Sub-task: Add client-side validation
- Sub-task: Display messages

Task: Backend – API & Business Rules
- Sub-task: Create transfer API
- Sub-task: Implement validation rules
- Sub-task: Update balances
- Sub-task: Log transactions

Task: Database Updates
- Sub-task: Schema changes (if any)
- Sub-task: Rollback mechanism

Task: Testing
- Sub-task: Unit tests
- Sub-task: Integration tests
- Sub-task: QA test scenarios

Writing Effective User Stories: Tips

Effective user stories are concise but meaningful. They are written collaboratively, refined regularly, and backed with acceptance criteria to avoid ambiguity.

- Keep them short and simple
- Focus on value, not tasks
- Collaborate with users to refine
- Add acceptance criteria for clarity

Good vs. Bad User Stories (Example 1)

Good user stories emphasize value for the user and are testable. Poor stories often sound like technical tasks without a clear user benefit.

- Good: clear, valuable, testable
- Bad: vague, technical, or incomplete
- Example:
  - ✅ “As a customer, I want to reset my password so I can regain access easily.”
  - ❌ “Add password reset function.”

Good vs. Bad User Stories (Example 2)

The difference lies in focus. Good stories capture intent and user benefit, while bad ones prematurely dictate technical details.

- Good: describes intent, not implementation
- Bad: overly detailed upfront
- Example:
  - ✅ “As a teacher, I want to upload lecture notes so students can study anytime.”
  - ❌ “Create a 20MB PDF upload system with 5 fields.”

User Story Mapping

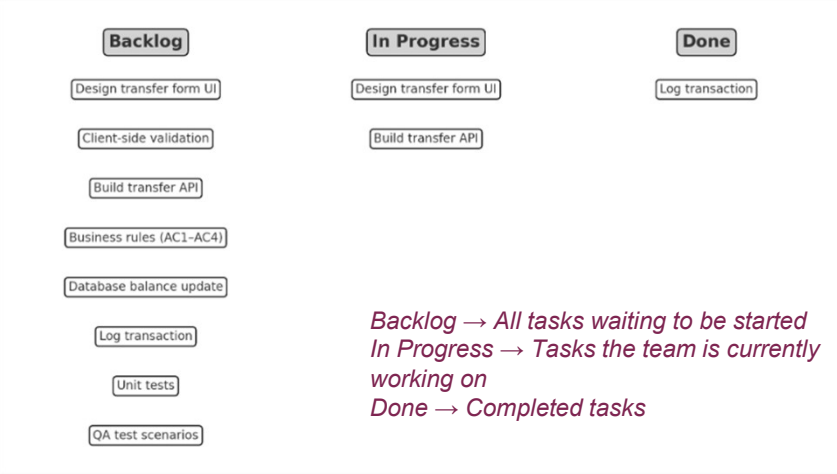
User story mapping turns stories into a structured visual map. It shows the big picture, highlights missing requirements, and guides release planning to maximize delivered value.

- Visual practice for **organizing work**
- Helps identify sets of high-value features
- Detects omissions in backlog
- Supports effective **release planning**

Roadmap to Backlog Breakdown

Story mapping also links high-level vision to actionable backlog items. By progressively breaking down roadmap features, teams ensure clarity and alignment between vision and tasks.

- Decompose roadmap into smaller requirements
- Use mapping & impact analysis
- Product Owner + team clarify expectations together



Examples of User Stories (Education App)

Educational apps benefit from user stories by capturing both student and teacher perspectives, ensuring features meet practical needs.

- As a student, I want online quizzes so that I can test myself anytime.
- As a lecturer, I want attendance tracking so that I can monitor participation.

Examples of User Stories (Banking App)

In banking, user stories focus on security and convenience. Each story highlights the value to both customers and staff.

- As a customer, I want instant balance check so that I can manage money better.
- As a bank manager, I want fraud alerts so that I can protect customers.

Classroom Activity: Writing User Stories (1/2)

This exercise encourages students to practice writing user stories. It reinforces how to capture user needs in simple, valuable statements.

- Form groups of 2–3
- Choose a system (e.g., food delivery, e-learning)
- Write 2 user stories per group
- Use collaboration-based approach with a building blocks styles

Classroom Activity: Writing User Stories (2/2)

By sharing and refining stories, students see how collaborative improvement makes stories clearer and more actionable, just like in real Agile projects.

- Share stories with class
- Refine using INVEST criteria
- Add acceptance criteria

Section Wrap-up: Why User Stories Matter

User stories are central to Agile because they ensure work always ties back to user value. With tools like story mapping, they help connect the big picture to daily tasks.

- Keep focus on user value
- Support collaboration & clarity
- Map vision → backlog → release
- Improve planning and adaptability

Assignment #1: User Stories

You are required to work in a team of four (4) to analyze the requirements of a **Contract Management System**.

1. High-Level User Story Development

- Write user story of this system using Collaboration-based approach
- Include relevant acceptance criteria to ensure clarity of the story.

2. Detailed User Story Breakdown (Jira-style)

- Break down your chosen high-level user story into tasks and sub-tasks similar to Jira structure.
- Each sub-task must be specific, actionable and tested (e.g., “Implement contract upload form”, “Validate contract expiry date”, “Write unit tests for contract search”).

Rubric Assignment #1: User Stories

◆ Marking Rubric (15 Marks, 4-Scale)

Criteria	Marks	Excellent (Full Marks)	Good (Mid-High)	Fair (Mid-Low)	Poor (Low/0)
High-Level User Story Development (C4 – Analyze)	3	Clear, well-structured story using collaboration-based approach (role, feature, benefit). Directly aligned with system requirements.	User story mostly clear; slight issues in format or alignment with requirements.	User story vague or missing one key element (role/feature/benefit).	Story unclear, incomplete, or not aligned with requirements.
Acceptance Criteria (C5 – Evaluate)	3	3–4+ acceptance criteria; specific, measurable, testable, and directly linked to user story.	Criteria mostly clear; some lack detail or testability.	Few criteria (1–2) OR criteria vague/general.	Missing, irrelevant, or unusable acceptance criteria.
Detailed User Story Breakdown (Tasks & Sub-tasks) (C6 – Create)	9	Comprehensive breakdown into logical tasks & sub-tasks (frontend, backend, DB, testing). Each sub-task is specific, actionable, and testable.	Good breakdown with clear tasks; minor gaps in coverage or detail.	Partial breakdown with limited structure (e.g., only frontend/backend); some tasks too broad.	Minimal or no breakdown; tasks unclear, incomplete, or not actionable.



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my